Contextual Bandit Event Recommendation for JoinIn

Marco Carraro Francesco Vezzani

Abstract

We build a reinforcement-learning event recommender for JoinIn, a Flutter/Firebase app where students find activities through a swipe feed. The existing Firestore query stays as a hard filter, and a Contextual Thompson Sampling re-ranker decides the order of the events that pass it. Ranking uses user, event, time, popularity, and safety features that the app already stores. User actions map to shaped rewards. For this class project we leave the remote logging hooks as no-ops and evaluate the policy offline with a synthetic simulator, instead of changing the Firebase backend.

1. Goal and Setting

JoinIn shows each user a short feed of candidate activities. Before this work, the feed was sorted mainly by event time, after dropping events that were expired, full, reported, owned by the user, or already joined. Our goal is to turn this fixed order into a learning recommender: one that favors events the user is likely to enjoy while still exploring uncertain ones.

We add the recommender as a re-ranking layer only. Firestore still retrieves the safe candidates, and the bandit just reorders the current slate. This keeps product risk low, because it never bypasses the existing business rules, moderation filters, or capacity limits.

2. RL Formulation

At each step the app sees a user u and a candidate event e that passed the filter. We describe the pair with a context vector

$$x_{u,e} = [\text{profile}(u), \text{event}(e), \text{time}(e), \text{social}(e), \text{safety}(e)].$$

The action is the order of events in the feed. The reward comes from what the user does next with the card.

. **AUTHORERR: Missing \icmlcorrespondingauthor.**

Proceedings of the 38^{th} International Conference on Machine Learning, PMLR 139, 2021.

Table 1. Reward shaping used by the implementation.

Action	Reward
Impression	0.0
Join free activity	1.0
Request to join	0.7
Maybe/save	0.3
Pass/swipe left	-0.1
Leave after join	-0.6
Report activity	-1.0
Chat activity after join	0.2

Algorithm 1 Feed re-ranking with Contextual Thompson Sampling

Input: user u, filtered events E
for each event $e \in E$ **do**
  Build feature vector $x_{u,e}$
  Sample weights $\tilde{\theta}$ around the current prior
  Compute score $s_e = x_{u,e}^T \tilde{\theta}$
end for
Sort events by s_e descending
Log one impression document per shown event
On user action, log action and shaped reward

3. Algorithm Choice

We use Contextual Thompson Sampling with a linear reward model (Agrawal & Goyal, 2013; Li et al., 2010):

$$\hat{r}(u, e) = x_{u,e}^T \theta.$$

For each candidate, the app draws a perturbed weight vector $\tilde{\theta}$ from a light posterior approximation and ranks by $x_{u,e}^T \tilde{\theta}$. High-scoring events go first, but uncertain yet plausible events still get a chance to appear.

We prefer this over a deep RL method because JoinIn has a changing event catalog, sparse feedback, and strict product rules. A contextual bandit fits well: each swipe is a single one-step decision, the candidate set turns over quickly, and a first deployment must stay interpretable and safe. The generic ranking step comes from the published Dart package `dart_rl` version 1.0.0 (Vezzani, 2026); the app only adds JoinIn-specific features, priors, and logging.

4. Implementation

4.1. Codebase Integration

We take the reusable linear Thompson Sampling code from the published `dart_rl` package and keep all product-specific logic in JoinIn:

- **Dependency**: `pubspec.yaml` adds `dart_rl: ^1.0.0`.
- **Reusable package**: `dart_rl` provides `LinearThompsonSampling`, `ContextualBanditArm`, and `ContextualBanditRanking`.
- **Recommendation service**: feature extraction, prior weights, reward mapping, and logging hooks for the local simulator.
- **Home feed**: runs the re-ranker after candidate retrieval and records feed interactions.

Candidate retrieval stays in `FirebaseApi.fetchEventsForFeed`. The query filters by future date, visible universities, ownership, participation, report threshold, user reports, and capacity. The re-ranker only ever sees candidates that already passed these checks.

4.2. State Features

The feature vector is small and uses only data already in `EventModel` and `UserModel`: a bias term, university match, event type, time-to-event, evening and weekend flags, attendee ratio, waiting-list ratio, image availability, freshness, profile-category match, and a report penalty.

```
static Map<String, double> buildFeatures({
  required UserModel user,
  required EventModel event,
}) {
  final attendeeCount = event.attendees?['uids']?.length ?? 0;
  final waitingCount = event.waitings?['uids']?.length ?? 0;
  final maxAttendees = event.maxAttendees?.toDouble();

  return {
    'bias': 1.0,
    'same_university': event.university == user.university ? 1.0 : 0.0,
    'free_to_join': event.type == EventType.freeToJoin.toString() ? 1.0 :
        0.0,
    'request_to_join': event.type == EventType.requestToJoin.toString() ?
        1.0 : 0.0,
    'attendee_ratio': _clamp01(attendeeCount / capacity),
    'waiting_ratio': _clamp01(waitingCount / capacity),
    'report_penalty': _clamp01((event.reportCount ?? 0) / 5.0),
  };
}
```

4.3. Ranking

The service holds an interpretable prior weight vector and one standard deviation per feature. On each feed refresh it hands the ranking step to `dart_rl`, which samples around the prior and sorts events by the sampled score.

```
static List<RankedEvent> rankEvents({
  required List<EventModel> events,
  required UserModel user,
}) {
  final bandit = LinearThompsonSampling<EventModel>(
    weights: _priorWeights,
    standardDeviations: _posteriorStd,
  );
  final arms = events.map((event) => ContextualBanditArm(
    item: event,
    features: buildFeatures(user: user, event: event),
  )).toList();
  return bandit.rank(arms).map((ranking) {
    return RankedEvent(
      event: ranking.item,
      score: ranking.score,
      exploitationScore: ranking.expectedReward,
      explorationBonus: ranking.explorationBonus,
      features: ranking.features,
    );
  }).toList();
}
```

4.4. No Server-Side Logging

Our early analysis showed that real pass and maybe signals live only in the local Isar store, and that production training would need centralized logs. For this class project we chose not to touch the Firebase backend or add new server collections. So the app-side logging methods are no-ops by default, and we evaluate with the local synthetic simulator included in the submission folder.

```
static Future<void> logInteraction({
  required UserModel? user,
  required EventModel event,
  required RecommendationAction action,
  required String sessionId,
}) async {
  debugPrint('Skipped remote logging; synthetic simulations are used.');
}
```

5. Evaluation Plan and Results

We checked that the implementation builds: `flutter pub get` resolves the published `dart_rl` 1.0.0 package from pub.dev, and the Flutter analyzer reports only pre-existing style hints in `home_page.dart`, not type errors from the recommender.

Because we do not change the Firebase server, evaluation runs through the local script `simulations/synthetic_bandit_simulation.py`. Training on real interactions would mean backend changes and a privacy-policy update, since we would collect and process new behavioral signals for personalization. That review takes longer than the class-project window, so we use synthetic data now and plan a privacy-reviewed release for Q4. The simulator creates 1,200 users with 30 candidate events each. Each policy shows the top five events per user, giving 6,000 impressions per policy. Users have a university and a preferred category; events have category, university, type, time, popularity, freshness, image availability, and report risk. A hidden reward model then picks each action using the same reward shaping as the app.

Table 2. Synthetic simulation results. Each policy is evaluated on 6,000 impressions. Delta is CTS minus baseline.

Metric	Baseline	CTS	Delta
Join rate	40.97%	58.10%	+17.13 pp
Request rate	15.43%	10.95%	-4.48 pp
Pass rate	43.60%	30.95%	-12.65 pp
Avg. reward/impression	0.474	0.627	+0.153

The baseline sorts candidates by earliest event time. The proposed method ranks the same candidates with Contextual Thompson Sampling. The run writes `results/synthetic_metrics.json` and `results/synthetic_interactions.csv`.

The bandit policy raises the join rate and the average reward and lowers the pass rate. The request rate drops because the learned ranking favors free-to-join events, which carry the strongest positive reward in the synthetic user model. That is fine for a first version, since a direct join is the best outcome, but a production system should watch that request-to-join events do not become under-exposed.

6. Limitations and Next Steps

The current model uses a fixed prior and on-device random exploration. This is safe for a first deployment and fits a class project that avoids server changes, but it does not yet update the posterior from real user rewards. The synthetic simulator checks the mechanics and the reward design; it cannot prove a production lift. The Q4 release should: update the privacy policy and backend data contract; add an opt-in analytics pipeline; estimate θ from real past interactions; and ship updated weights through Remote Config or a server endpoint. It should also model slate effects and position bias, since an event's reward depends on where it sits in the stack.

References

Agrawal, S. and Goyal, N. Thompson sampling for contextual bandits with linear payoffs. In *Proceedings of the 30th International Conference on Machine Learning*, pp. 127–135, 2013.

Li, L., Chu, W., Langford, J., and Schapire, R. E. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pp. 661–670, 2010.

Vezzani, F. dart_rl: Reinforcement learning algorithms for dart. https://pub.dev/packages/dart_rl, 2026. Version 1.0.0, published on pub.dev.